Analysis and Implementation of PageRank for General Directed Graphs

Dario Pomasqui[1]

Yachay Tech University, School of Mathematical and Computational Sciences, Ecuador
dario.pomasqui@yachaytech.edu.ec

Abstract. This paper presents the final version of an Analysis of Algorithms project on PageRank, a classic graph algorithm used to assign relative importance scores to the vertices of a directed graph. The first version of the project focused on a small example implemented in Python. The final version generalizes the problem to directed graphs of arbitrary size, reads input files as edge lists, and implements the algorithm in C++ using inverse adjacency lists. The report includes a formal definition of the algorithm, implementation details, examples of problems it solves, mathematical analysis, best-case, average-case, and worst-case complexity, and empirical results obtained with synthetic datasets. The experiments show stable convergence for graphs from 100 to 5000 nodes and confirm that, with a sparse representation, the cost per iteration is linear in the number of vertices and edges.

Keywords: PageRank · graph algorithms · power iteration · computational complexity · C++.

1 Introduction

PageRank was originally proposed as part of the Google search engine to rank web pages according to their structural importance in the hyperlink graph [2, 1]. Although its first motivation came from the Web, the algorithm is also useful for citation networks, software dependencies, recommendation systems, social networks, and other domains where relationships have direction.

The main idea is that a node is important if it receives links from other important nodes. This definition is recursive, so simply counting incoming links is not enough. PageRank solves the problem by modeling the behavior of a random surfer. At each step, the surfer follows an outgoing link with high probability or jumps to a random node with a small probability. The stationary distribution of this process defines the score of each node. This view belongs to a broader family of link-analysis methods, including HITS, which also uses hyperlink structure to identify authoritative sources [3].

In the first project presentation, PageRank was implemented in Python for a small graph. That version was useful for understanding the transition matrix, dangling nodes, convergence, and validation against NetworkX. However, a final

project for Analysis of Algorithms should not be limited to a manual example. For this reason, this paper presents a more general C++ implementation that reads external datasets and processes larger sparse directed graphs.

The general objective of the project is to study PageRank from three perspectives: mathematical formalization, efficient implementation, and empirical analysis. The specific objectives are:

- to define PageRank formally on directed graphs;
- to implement the algorithm in C++ without relying on external graph libraries;
- to handle dangling nodes correctly;
- to analyze best-case, average-case, and worst-case complexity;
- to measure running time, iterations, and convergence error;
- to provide a reproducible base for the final presentation.

The main contribution of the final version is the transition from a didactic dense-matrix prototype to a reusable sparse-graph implementation. This is important because many real graphs are sparse: the number of existing edges is much smaller than the number of possible edges. Storing a full matrix in such cases wastes memory and makes the algorithm less scalable.

2 Formal Definition of the Problem

Let $G = (V, E)$ be a directed graph, where V is the set of vertices and $E \subseteq V \times V$ is the set of directed edges. If $(u, v) \in E$, then there is a link from u to v. Let $n = |V|$ be the number of vertices, $m = |E|$ the number of edges, $out(u)$ the out-degree of vertex u, and $In(v)$ the set of predecessors of vertex v.

The goal of PageRank is to compute a vector r such that each component $r(v)$ represents the relative importance of vertex v. The vector must satisfy two conditions: all values are nonnegative and the total sum is one. Therefore, PageRank produces a probability distribution over the graph vertices.

The equation used in this project is:

$$r(v) = \frac{1-d}{n} + d \sum_{u \in In(v)} \frac{r(u)}{out(u)} + d\frac{D}{n}, \tag{1}$$

where d is the damping factor and D is the total PageRank mass stored in dangling nodes, that is, nodes with no outgoing edges. The most common value for d is 0.85. If d is small, the ranking becomes closer to a uniform distribution. If d is large, the ranking depends more strongly on the graph structure, but convergence may require more iterations.

2.1 Dangling Nodes

A dangling node is a vertex with no outgoing edges. In the random surfer model, if the surfer reaches a node with no outgoing links, there is no link to follow.

If the algorithm ignored this case, probability mass would disappear during the iteration. To avoid this problem, the implementation distributes the total mass of all dangling nodes uniformly among all vertices.

2.2 Probabilistic Interpretation

PageRank can be interpreted as a Markov chain. At each step, the random surfer follows an outgoing edge with probability d, or performs a random jump with probability $1-d$. The random jump is called teleportation. This term is essential because it allows the process to leave closed components and gives every node a positive probability of receiving rank mass.

3 Design Paradigm

PageRank follows an iterative design paradigm. It does not compute a closed-form solution directly. Instead, it starts with a uniform distribution and repeatedly updates the vector until two consecutive iterations are sufficiently close. This method is a form of power iteration, a standard approach for link-analysis problems at web scale [7].

The algorithm can also be understood as a fixed-point method. The PageRank equation defines a transformation $F(r)$. The goal is to find a vector r such that $F(r) = r$. In practice, the algorithm computes $r^{(1)}, r^{(2)}, r^{(3)}$, and so on, until a stopping criterion is reached.

The stopping criterion used in this project is:

$$\sum_{v \in V} |r_{new}(v) - r_{old}(v)| < \varepsilon. \tag{2}$$

The experimental tolerance was $\varepsilon = 10^{-10}$. A maximum number of iterations is also used to avoid long executions in extreme parameter configurations.

4 Algorithm

The algorithm receives a directed graph and returns a list of nodes sorted by PageRank score. The program input is a CSV file with columns `source,target`. Each row represents one directed edge.

```
Input: directed graph G=(V,E), damping d, tolerance eps
Initialize rank[v] = 1 / |V| for every v in V
for iter = 1 to max_iter:
    dangling = sum rank[u] for all u with out_degree[u] = 0
    base = (1 - d) / |V| + d * dangling / |V|
    next[v] = base for every v
    for each vertex v:
        for each predecessor u of v:
```

```
            next[v] += d * rank[u] / out_degree[u]
    error = sum |next[v] - rank[v]|
    rank = next
    if error < eps:
        stop
return rank
```

The output is another CSV file with columns `rank`, `node`, `score`, `out_degree`, and `in_degree`. This makes it possible to inspect not only the final score, but also the relation between PageRank and node degree.

5 Implementation Details

The final implementation is written in C++17. The main program is `pagerank.cpp`. A second program, `generate_dataset.cpp`, was implemented to generate synthetic directed graphs for empirical experiments.

5.1 Graph Representation

An important decision was to avoid a dense $n \times n$ matrix. In real sparse graphs, most entries of such a matrix would be zero. Storing the entire matrix would waste memory. The final version uses inverse adjacency lists: for each node v, the program stores the list of nodes that point to v. This representation matches the PageRank equation directly, because the score of v is computed from its predecessors.

The implementation also stores the out-degree of every node. If a predecessor u points to many nodes, its individual contribution to each target is smaller because its rank mass is divided among all outgoing edges. Therefore, $out(u)$ is required in every iteration.

5.2 Input Processing

The program accepts arbitrary node labels. For example, nodes may be named `A`, `N120`, or `paper45`. Internally, these labels are mapped to integer indices using a hash table. This separates the external dataset format from the efficient internal representation.

An example input file is:

```
source,target
A,B
A,C
B,C
C,A
```

The program also removes duplicate edges and ignores self-loops in the synthetic generator. This keeps the graph representation clean and avoids counting the same directed relation more than once.

5.3 Parameters

The executable accepts command-line parameters:

```
pagerank --input edges.csv --output scores.csv
         --damping 0.85 --tol 1e-10 --max-iter 300
```

The damping factor controls how much weight is assigned to links. The tolerance controls the precision of the stopping criterion. The maximum number of iterations protects the program from configurations that converge slowly.

5.4 Dataset Generator

For the empirical part, a synthetic graph generator was implemented. The user specifies the number of nodes, the desired average degree, and a seed. The generator produces a CSV file with directed edges. It first includes a simple chain to ensure basic connectivity and then adds random edges until the target number of edges is reached.

6 Problems Solved by PageRank

PageRank solves ranking problems in directed networks. Its original application was web search, where each page is a node and each hyperlink is a directed edge. However, the same model can be applied in several other domains:

- citation networks, where an important paper is cited by other important papers;
- software dependency graphs, where central packages are used by many projects;
- recommendation systems, where users, products, or documents form directed relations;
- social network analysis, where following relationships have direction;
- internal document ranking for repositories, wikis, and organizational knowledge bases.

This project uses two types of data. First, a small graph called `web_example.csv` is used for manual inspection of the output. Second, synthetic graphs with 100, 500, 1000, and 5000 nodes are used to evaluate scalability.

7 Mathematical Analysis

The convergence of PageRank is based on teleportation. If the surfer only followed links, the process could become trapped in a closed component of the graph. The term $(1 - d)/n$ prevents this problem because every node receives a small amount of probability in each iteration. For this reason, the resulting Markov chain has a unique stationary distribution [4, 5].

7.1 Probability Invariant

At every iteration, the PageRank vector must remain a probability distribution. This means that the sum of all scores must be one. Teleportation contributes a total mass of $1 - d$. Link contributions and dangling-node redistribution contribute a total mass of d. Therefore, the total mass is:

$$(1 - d) + d = 1. \tag{3}$$

All terms in the equation are nonnegative, so no score becomes negative during the execution.

7.2 Effect of the Damping Factor

The damping factor d affects both the final ranking and the convergence speed. When d is low, the algorithm is closer to a uniform distribution and converges faster. When d is high, the graph structure has more influence, but more iterations are usually required.

This behavior appears in the experiments. For a synthetic graph with 1000 nodes and 6000 edges, using tolerance 10^{-10}, the results in Table 1 were obtained.

Table 1. Effect of the damping factor on a graph with 1000 nodes.

Damping factor d	Iterations	Final error
0.50	16	$2.55 \cdot 10^{-11}$
0.75	21	$7.11 \cdot 10^{-11}$
0.85	24	$5.69 \cdot 10^{-11}$
0.90	25	$9.11 \cdot 10^{-11}$
0.95	27	$6.61 \cdot 10^{-11}$

8 Complexity Analysis

Let n be the number of vertices, m the number of edges, and k the number of iterations until convergence. If a dense matrix were used, each iteration would cost $O(n^2)$. This is not appropriate for large sparse graphs.

The final implementation scans all vertices to initialize the next vector and all edges to accumulate link contributions. Therefore, the cost per iteration is:

$$O(n + m). \tag{4}$$

The total running time is:

$$O(k(n + m)). \tag{5}$$

The memory usage is also $O(n + m)$, because the implementation stores labels, degrees, predecessor lists, and two PageRank vectors.

Table 2. Complexities of the final implementation.

Case	Time	Explanation
Best case	$O(n+m)$	Convergence after one iteration.
Average case	$O(k(n+m))$	Convergence after k iterations.
Worst case	$O(K(n+m))$	The maximum number K of iterations is reached.
Memory	$O(n+m)$	Sparse graph and auxiliary vectors.

9 Empirical Analysis

The experiments were executed with C++ compiled using the `-O2` optimization flag. The purpose was to verify that the implementation converges and that it can process graphs larger than the manual example used in the first version.

Table 3. Empirical results with $d = 0.85$ and tolerance 10^{-10}.

Dataset	Nodes	Edges	Iterations	Final error
web example	20	30	44	$8.41 \cdot 10^{-11}$
synthetic 100	100	600	22	$8.48 \cdot 10^{-11}$
synthetic 500	500	3000	23	$5.86 \cdot 10^{-11}$
synthetic 1000	1000	6000	24	$5.69 \cdot 10^{-11}$
synthetic 5000	5000	30000	24	$5.58 \cdot 10^{-11}$

All datasets converged with the specified tolerance. The number of iterations remained low for the synthetic graphs, which shows that the algorithm is practical for these input sizes. The absolute running times were small because the datasets are moderate and the implementation was compiled with optimization.

9.1 Interpretation of Results

Table 3 shows that the algorithm is no longer tied to a small graph. The program processes external datasets and produces output files sorted by PageRank. Increasing the number of edges from 600 to 30000 does not change the structure of the algorithm: each iteration scans vertices and edges once.

The small manual graph requires more iterations than several larger synthetic graphs. This is possible because convergence speed does not depend only on input size. It also depends on graph structure, connected components, degree distribution, and dangling nodes.

9.2 Limitations

The synthetic datasets do not fully represent the real Web. They do not necessarily contain heavy-tailed degree distributions, natural communities, or realistic citation patterns. Therefore, the results should be interpreted as an initial evaluation of complexity and functionality, not as an industrial-scale performance study.

10 Implementation Validation

The implementation was validated at two levels. First, it was checked that the program preserves PageRank mass close to one after each iteration. This property is important because PageRank represents a probability distribution. If the sum moved away from one, the ranking would lose its probabilistic interpretation.

Second, the final version was compared conceptually with the first Python version. The first version used an explicit transition matrix and allowed manual inspection of stochastic columns. The C++ version does not build the full matrix, but it applies the same equation. Therefore, both versions share the same mathematical basis: uniform initialization, damping factor, uniform handling of dangling nodes, and an L_1-norm stopping criterion.

10.1 Test Cases

The first test case was the `web_example.csv` dataset. This file contains 20 nodes and 30 edges. Its purpose is not performance measurement, but small-scale inspection of the output. In this case, the algorithm converged in 44 iterations.

The second group of tests used synthetic graphs. These graphs allow control over the number of nodes and the average degree. This makes it possible to observe how the cost changes as the input size grows. Although the graphs are not real web graphs, they are useful for evaluating algorithmic behavior.

10.2 Success Criteria

The implementation is considered correct for the purposes of the course if it satisfies the following conditions:

- it reads an edge list from a file;
- it includes all referenced nodes;
- it removes duplicate edges;
- it handles nodes with no outgoing edges;
- it produces nonnegative scores;
- it converges with the established tolerance;
- it exports the final ranking reproducibly.

All these criteria were satisfied in the executions stored in the `results` folder.

11 Comparison Between the First Version and the Final Version

Table 4 summarizes the main differences between the first project version and the final version.

The first version was useful for learning the algorithm. It allowed matrix visualization, convergence plots, and comparison with a known library. The final

Table 4. Comparison between project versions.

Aspect	First version	Final version
Language	Python	C++17
Input	Graph defined in code	CSV file
Structure	Dense matrix	Predecessor lists
Scale	Small graph	General graphs
Validation	NetworkX	Reproducible experiments
Output	Notebook tables	CSV files

version is oriented toward a complete Analysis of Algorithms project: it separates code, data, results, and report, and it allows experiments to be repeated from the terminal.

The most important difference is the data structure. In the first version, a dense matrix was acceptable because the graph was small. In the final version, a dense matrix would be unnecessary and expensive. For a graph with 5000 nodes, a dense matrix would contain 25 million entries, while the dataset used in this project contains only 30000 edges. The sparse representation avoids storing most zero entries.

12 Relevant Implementation Fragments

The following fragment represents the main PageRank update. First, it computes dangling-node mass. Then it initializes each node with teleportation and dangling contribution. Finally, it adds the contributions from predecessors.

```
for (int iter = 1; iter <= max_iterations; ++iter) {
    double dangling_mass = 0.0;
    for (int u = 0; u < n; ++u) {
        if (graph.out_degree[u] == 0) {
            dangling_mass += rank[u];
        }
    }

    const double dangling_contribution =
        damping * dangling_mass / n;
    fill(next.begin(), next.end(),
         teleport + dangling_contribution);

    for (int v = 0; v < n; ++v) {
        double link_sum = 0.0;
        for (int u : graph.incoming[v]) {
            link_sum += rank[u] / graph.out_degree[u];
        }
        next[v] += damping * link_sum;
```

```
    }
}
```

The dataset generator is also important for reproducibility. It creates random edges using a fixed seed. By using the same seed, another student or the instructor can generate the same graph and check the results.

```
while (edges.size() < target_edges) {
    int u = dist(rng);
    int v = dist(rng);
    if (u != v) {
        edges.insert({u, v});
    }
}
```

These fragments show that the project does not depend on an external library to compute PageRank. The core logic is implemented directly and corresponds to the formal definition presented in this report.

13 Reproducibility

For an algorithmic experiment to be useful, it must be reproducible. For this reason, the project separates source code, datasets, executables, and results. The file `run_experiments.ps1` compiles the programs, generates the datasets, and runs PageRank on each one.

Reproducibility also depends on fixed seeds. In the synthetic datasets, different seeds were used for each graph size, but the seeds remain constant across executions. This means that if the script is run again, the same input files are produced. Running times may vary slightly because of operating system conditions, but the edges and rankings should remain consistent.

The main result of each execution is stored in CSV files. This makes it easy to inspect the rankings, create report tables, or generate plots in another environment. For example, `synthetic_1000_scores.csv` contains the ranking for the 1000 nodes in the corresponding synthetic dataset.

14 How to Run the Project

From PowerShell, the project folder is opened with:

```
cd C:\Users\User\Desktop\final\VersionFinal_PageRank
```

To compile:

```
g++ -O2 -std=c++17 src\pagerank.cpp -o pagerank.exe
g++ -O2 -std=c++17 src\generate_dataset.cpp -o generate_dataset.exe
```

To run PageRank on the small example:

```
.\pagerank.exe --input data\web_example.csv
               --output results\web_example_scores.csv
```

To generate a new dataset:

```
.\generate_dataset.exe --output data\my_graph.csv
                       --nodes 2000 --avg-degree 6 --seed 2026
```

To run all experiments:

```
powershell -ExecutionPolicy Bypass -File .\run_experiments.ps1
```

15 Discussion

The main improvement over the first version is generalization. The first submission made it possible to understand the algorithm step by step, but it was limited by dense matrices and small examples. The final version uses a sparse representation in C++, reads external files, and produces exportable results.

Another improvement is the connection between code and analysis. The predecessor list is not only a programming decision; it appears directly from the PageRank equation. The treatment of dangling nodes is also essential because it preserves probability mass.

The current implementation could be extended with parallelization, real datasets, compressed graph storage, and comparison against optimized libraries. Another possible extension is personalized or topic-sensitive PageRank, where teleportation is not uniform but depends on a preference vector or on a set of topic distributions [6].

16 Conclusions

PageRank is an important algorithm because it combines graph theory, probability, linear algebra, and complexity analysis. In this project, a general C++ version was implemented for directed graphs, with support for external datasets, dangling nodes, and configurable convergence criteria.

The theoretical analysis shows that the sparse implementation has per-iteration cost $O(n+m)$, while a dense matrix implementation would have cost $O(n^2)$. The empirical analysis confirms that the program converges on the evaluated datasets and can process graphs much larger than the initial example.

The final version satisfies the required project elements: formal definition, C++ implementation, examples of problems it solves, datasets, mathematical analysis, best-case, average-case, and worst-case complexity, and empirical results.

Annexes

Overleaf link: add the link here after uploading the manuscript.

GitHub link: add the link here after uploading the implementation.

Main files: `src/pagerank.cpp`, `src/generate_dataset.cpp`, `data/`, `results/`, `run_experiments.ps1`.

Acknowledgments. This work was developed as a final project for the Analysis of Algorithms course.

Disclosure of Interests. The author declares no competing interests related to the content of this project.

References

1. Page, L., Brin, S., Motwani, R., Winograd, T.: The PageRank Citation Ranking: Bringing Order to the Web. Technical Report, Stanford InfoLab (1999). https://ilpubs.stanford.edu/422/
2. Brin, S., Page, L.: The Anatomy of a Large-Scale Hypertextual Web Search Engine. Computer Networks and ISDN Systems **30**(1–7), 107–117 (1998). https://doi.org/10.1016/S0169-7552(98)00110-X
3. Kleinberg, J.M.: Authoritative Sources in a Hyperlinked Environment. Journal of the ACM **46**(5), 604–632 (1999). https://doi.org/10.1145/324133.324140
4. Langville, A.N., Meyer, C.D.: Deeper Inside PageRank. Internet Mathematics **1**(3), 335–380 (2004). https://doi.org/10.1080/15427951.2004.10129091
5. Langville, A.N., Meyer, C.D.: Google's PageRank and Beyond: The Science of Search Engine Rankings. Princeton University Press, Princeton (2012)
6. Haveliwala, T.H.: Topic-Sensitive PageRank. In: Proceedings of the Eleventh International World Wide Web Conference, pp. 517–526 (2002). https://ilpubs.stanford.edu/573/
7. Leskovec, J., Rajaraman, A., Ullman, J.D.: Mining of Massive Datasets. 3rd edn. Cambridge University Press, Cambridge (2020). https://www.mmds.org/
8. Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.: Introduction to Algorithms. 4th edn. MIT Press, Cambridge (2022)
9. CEDIA: Scientific Track - TICEC 2026, https://ticec2026.cedia.edu.ec/track-cientifico/, last accessed 2026/04/29